SafeGrid-AI: An Explainable Multi-Agent System for Dynamic Power Routing

Disha Kaushal
Roll No. 23K91A0563
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
[CITY]

Bollam Rakesh
Roll No. 23K91A0542
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
[CITY]

Chinnolla Ravindhar
Roll No. 23K91A0554
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
[CITY]

Affan Mohammed Ramy
Roll No. 23K91A0505
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
[CITY]

Chandrashekar (Guide)
[DESIGNATION]
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
[CITY]

***Abstract*---**Language-model agents can plan energy dispatch from a structured view of the grid, but a non-deterministic proposer that acts on physical infrastructure can request set-points that break voltage, line or battery limits. SafeGrid-AI is a neuro-symbolic multi-agent system for a low-voltage feeder with solar, batteries and an electric vehicle. Three agents propose one structured action per step; a coordinator orders them; a deterministic Symbolic Safety Shield validates every action against ten ordered checks, executing it unchanged, scaled down by bisection to the largest safe value, or not at all, and writes each decision to a hash-chained log with a structured explanation. On a pandapower AC model, 7 scenarios, 3 strategies and 3 seeds (63 runs), the shield approved 1,162, projected 237 and rejected 113 of 1,512 proposals and no executed action created or worsened a limit violation in 504 steps, whereas the same agents without it did so in 99 steps. Safety costs time (mean step 348 ms against 106 ms) and some solar energy. Every agent result uses a deterministic mock model, the guarantee is relative to the simulated model, and grid stress that exists with all agents idle is not repaired.

***Index Terms*---**agentic AI, explainability, large language models, multi-agent systems, neuro-symbolic safety, safety shielding, smart grid.

I. INTRODUCTION

Distributed solar, home batteries and electric vehicles turn the last kilometre of the distribution grid into a place where supply and demand must be balanced every few minutes and where a badly chosen set-point can push a voltage or a cable past its limit [2], [3]. Multi-agent systems are a natural fit, because each prosumer or device can own its decisions [4], [5]. Recent work lets large language models act as such agents: Grid-Agent plans remedial actions with power-flow solvers [6], and a Model Context Protocol server exposes a grid simulator to agents under human oversight [7]. Language models can reason over messy context and use tools [8], but their output is probabilistic. A hallucinated or over-sized command is harmless in text and unacceptable on a feeder.

Three properties are wanted at once: adaptability, so that decisions follow weather, price and demand; auditability, so that an operator can ask why a set-point was chosen; and a hard stop, so that no proposal can leave the permitted operating region. Fixed rules give the third property only if they are right for every case, and Section IV shows a rule controller that is not. Language-model agents give the first two but not the third. A design that keeps all three has to treat the proposer as untrusted.

The safe-learning literature offers a remedy: a shield that monitors every action and corrects it before it reaches the environment [9], or a safety layer that projects an action onto the feasible set [10], [11]. This paper brings that idea to language-model energy agents. We keep the agents free to propose, and put an independent, deterministic Symbolic Safety Shield between every proposal and the grid, so that safety does not depend on the model behaving well [12].

The contributions are: (1) a neuro-symbolic architecture in which an untrusted proposer, a coordinator, a shield and an AC power-flow simulator are separate components; (2) a shield that validates ten ordered checks, attributes only new or worse violations to an action, and projects an unsafe action onto the largest safe fraction by bisection; (3) a hash-chained audit log and structured, per-decision explanations; and (4) a reproducible comparison of three strategies over seven scenarios and three seeds, with the cost of safety and the limits of the evidence stated. Section II reviews related work, Section III describes the system, Section IV reports the measurements and Section V concludes.

II. RELATED WORK

Base paper. Eyimaya and Altin [1] design four rule-based agents (distributed energy resources, energy storage, load and grid) for a DC microgrid with PV, wind, a synchronous generator, a battery and critical and non-critical loads. In MATLAB/Simulink with a 20 microsecond sampling time and an 800 V bus, the bus voltage stays regulated and critical loads stay supplied in all eight operating scenarios. The authors name scalability, software integration, cybersecurity and the lack of experimental validation as open issues. The agents contain no language model, no formal safety layer and no explanation interface; those are the three things this work adds.

LLM agents for grids. Grid-Agent [6] pairs a planning agent with a validation agent that executes in a sandbox with rollback; on IEEE 69-bus, CIGRE MV and IEEE 30-bus networks its best language-model configurations report a 100 % success rate, weaker ones 63 % and 44 %. Safety there

comes from sandboxed trial and rollback rather than from a separate rule layer. Picault and Goubet [7] argue for traceable, stateful, human-supervised agent workflows and present a first MCP interface, without quantitative results. McArthur et al. [4] and Wooldridge [5] give the general case for agents in power engineering; Lasseter [3] and Farhangi [2] frame the microgrid and smart-grid setting.

Optimisation baselines. Classical optimal power flow is mathematically robust but, for modern grids with renewable generation and many discrete assets, becomes a mixed-integer, non-convex, unbalanced AC problem that is hard to solve [6]. The shield does not optimise anything: it only filters what an agent proposes, which keeps its cost to a few trial power flows per action and leaves the choice of objective, cost, self-consumption or something else, to the agent.

Safe learning and explanation. Shielding [9], safety layers [10] and safe reinforcement learning [11] constrain a learner's actions; Amodei et al. [12] list safe exploration and robustness among the basic problems of AI safety. For explanations, written reasoning from a language model can be unfaithful to what actually drove its answer [13], so our explanations are built from the recorded inputs, the chosen action and the shield's checks, not from model-written reasoning. Table I summarises the comparison.

TABLE I
COMPARISON WITH THE CLOSEST WORK

Work	Decision layer	Safety mechanism	Evidence
[1]	Four rule-based agents	Control logic regulates the bus; no validation layer	8 Simulink scenarios, DC microgrid
[6]	LLM planner + validator agent	Sandboxed trial with rollback	3 IEEE/CIGRE networks; 100 % best, 63 % and 44 % weaker
[7]	LLM agents via MCP, human in the loop	Human validation, traceability	Position paper, no results
This work	3 agents, pluggable LLM provider	Deterministic shield: approve, project, reject; audit chain	63 runs, 504 shielded steps, mock LLM

Positioning. Compared with [1], this work moves from a DC microgrid in Simulink to an AC feeder solved with pandapower, replaces fixed rules by a pluggable proposer, and adds a validation layer and an audit trail. Compared with [6], it validates an action before it is committed, deterministically and without a language model, and it attributes violations to actions rather than to the grid. Compared with [7], it supplies the auditable action path that paper calls for, but it does not address transmission-system studies. The comparison with [1] is qualitative: the two systems share no benchmark, so no like-for-like number exists.

III. PROPOSED SYSTEM AND METHOD

A. Architecture

The system (Fig. 1) is a React dashboard over a FastAPI service with REST routes and a WebSocket stream. A simulation service owns one engine per run. At each 15-minute step the engine takes seeded solar, load and price profiles, computes the no-action power flow with pandapower [14], lets each agent perceive a structured context, collects one proposal per agent, has the coordinator order them, validates them one by one against the cumulative grid state, commits only the validated set-points, and stores and streams the result. The agents and the language model are untrusted; the coordinator, shield and simulator are trusted, and a test enforces that the shield imports nothing from the language-model, agent, simulation or grid packages.

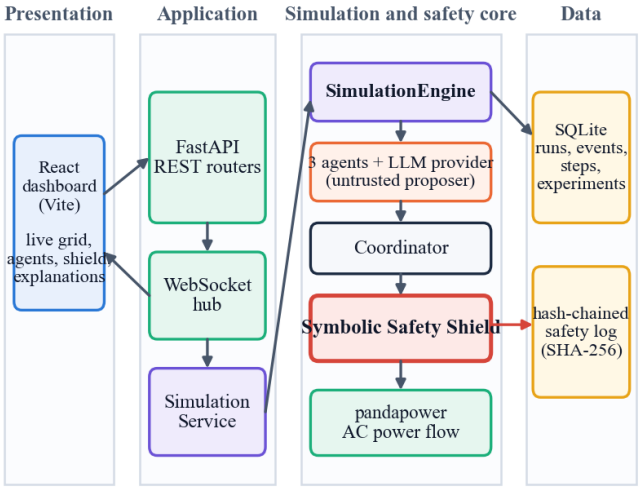


Fig. 1. System architecture. The proposer is untrusted; the coordinator, shield and power-flow simulator are deterministic trusted code.

B. Feeder, agents and proposals

The simulated feeder is a 20 kV source behind a 0.25 MVA 20/0.4 kV transformer with three low-voltage buses (NAYY 4x50 cables of 0.30, 0.45 and 0.25 km) and power factor 0.95. Three agents sit on it: a residential prosumer (6 kWp PV, 10 kWh battery), an electric vehicle (40 kWh, 7.4 kW, charge only) and a renewable prosumer (20 kWp PV, 30 kWh battery). An agent's context holds its load, solar output, battery state, price, its neighbours' exchange and voltage, and the grid limits. Its reply must be one EnergyAction: one of seven types (charge, discharge, import, export, curtail, shift load, idle), a finite power, a duration, a target and reason codes; unknown fields are rejected. A provider failure, timeout or malformed output becomes a labelled deterministic fallback decision that is shielded like any other. All measurements use a deterministic, seeded mock provider built on a cost-aware heuristic with seeded over-sizing; an API provider for Anthropic and OpenAI wire formats exists but was not run against a live model.

C. Coordinator

The coordinator flags three conflict types: peer mismatch, target contention and shared feeder. It orders proposals relief-first (idle, curtail and shift-load actions before charging and exporting), then in a rotating order, so that actions that lower stress are validated before actions that add it.

D. Symbolic Safety Shield

The shield (Fig. 2) reads only the structured action. Its ten ordered checks are structure, power limits, battery, node limits, power balance, power-flow convergence, bus voltage

(0.95 to 1.05 pu, tighter than the 10 % band of EN 50160 [15]), line loading (100 %), transformer loading (100 %) and import/export limits (200 kW). A violation is blamed on an action only if it is new or worse than in the all-idle grid state, so pre-existing stress does not cause good actions to be rejected. An action with no such violation is approved unchanged. Otherwise the shield projects it: with a the requested set-point and N(·) the set of new or worse violations,

$$\lambda^* = max\ \{\lambda : 0 \le \lambda \le 1,\ N(\lambda a) = \emptyset\}, \qquad (1)$$

found by bisection (40 iterations on local limits, 14 on power-flow limits). The rounded action is re-verified up to four times with a shrinking factor; a result below 0.05 kW, or one that flips an import into an export, is rejected. The shield never extends or enlarges a proposal.

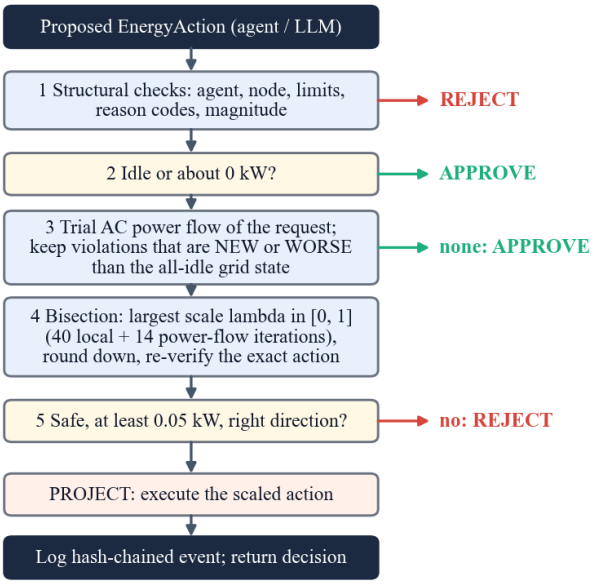


Fig. 2. Decision flow of the Symbolic Safety Shield for one proposed action.

E. Audit trail and explanation

Every non-approved action and every fallback is written as a safety event whose SHA-256 hash [16] covers the previous hash, the event identity and the payload, so removing or editing an event breaks the chain; verification passed in 21 of 21 shielded runs. Each decision also carries a structured explanation (inputs, decision factors, chosen action, shield result). No chain-of-thought is requested or stored, and a test checks this.

F. Implementation

The backend is 3,117 lines of Python (FastAPI 0.143, pydantic 2, pandapower 3.5.6) with a SQLite store of five tables (runs, events, safety events, steps, experiments); the tests add 624 lines and the dashboard 6,989 lines of JavaScript and CSS. REST routes start, stop, step and reset a run, run a scenario to completion, list agents and their decisions, read safety events and constraints, and launch comparison jobs; a dry-run route validates an operator-supplied action against the shield without applying it. The WebSocket stream carries agent actions, safety interventions, conflicts and grid updates, and the dashboard shows the live grid, each agent's decision with its explanation, the safety log and the power-flow charts. The API has no authentication because it is a single-user research prototype, the dashboard sign-in is a browser-side gate only, and model output is never executed as code, SQL or a shell command.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Setup

Seven seeded scenarios (normal, high solar, peak demand, battery stress, line congestion, voltage violation and unsafe AI action, which injects over-sized and malformed proposals) were run for 24 steps of 15 minutes under three strategies and seeds 0, 1 and 2, giving 63 runs. The strategies are a non-LLM time-of-use rule controller with no shield, the three agents with no shield (proposals applied as requested) and SafeGrid-AI (agents plus shield). The main metric counts the steps whose executed actions caused a violation that was new or worse than the all-idle grid state. All data are synthetic, generated from the seed, and the tariff is illustrative. The runs took about ten minutes on Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, no GPU used, with Python 3.11, pandapower 3.5.6 and FastAPI 0.143.

Metrics. Violations caused by actions counts limit violations that are new or worse than in the all-idle grid state, which is what the shield is designed to prevent. Steps with any violation after execution also include stress that no action caused. Raw-proposal violations are a counterfactual: the violations the unvalidated proposals of the same step would have caused. Renewable utilisation is one minus the curtailed share of the available PV energy (undefined in line congestion, where no PV energy is available), battery throughput is charged plus discharged energy divided by installed capacity, and step time is the wall-clock time of one step including agent decisions, validation and the power flow.

TABLE II
MEASURED RESULTS, MEAN PER RUN OVER 7 SCENARIOS AND 3 SEEDS (21 RUNS PER STRATEGY, 24 STEPS)

Metric	Rule baseline	Agents, no shield	SafeGrid-AI
Steps with violations caused by actions (of 24)	2.19	4.71	0.00
Violations caused by actions	2.19	6.86	0.00
Steps with any violation after execution	6.05	6.38	2.48
Voltage-violation steps	4.33	4.43	3.14
Line-overload events	1.71	1.86	0.71
Energy cost (INR, illustrative tariff)	216.4	380.8	225.0
Renewable utilisation	1.000	0.884	0.860
Battery throughput / capacity	0.520	3.392	0.508
Mean step time (ms)	107.9	106.2	347.8

B. Safety outcome

Table II and Fig. 3 give the main result. With the shield, executed actions caused no violation in any of the 504 steps (0.00 in every scenario). The same agents without it caused violations in 99 steps (144 violations in total), and the rule controller, although not a language model, caused them in 46 steps, all in line congestion (12.0 of 24 steps per run) and

peak demand (3.3); so a conventional controller needs a shield as well. Without the shield, the raw proposals of the shielded arm would have caused 192 violations in 145 steps.

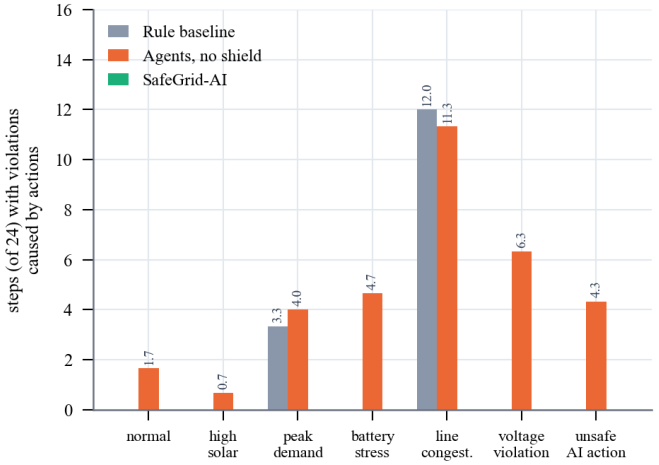


Fig. 3. Steps (of 24) with violations caused by executed actions, mean of three seeds, by scenario and strategy.

Per scenario (Fig. 3), the unshielded agents cause violations in all seven: 1.67 steps in normal, 0.67 in high solar, 4.00 in peak demand, 4.67 in battery stress, 11.33 in line congestion, 6.33 in voltage violation and 4.33 in unsafe AI action. The rule controller is safe in five scenarios but causes 12.00 steps in line congestion and 3.33 in peak demand, because it follows time-of-use rules and has no network check.

C. Shield behaviour

Of 1,512 proposals the shield approved 1,162 (76.9 %), projected 237 (15.7 %) and rejected 113 (7.5 %), logging 354 safety events and 241 coordinator conflicts. Fig. 4 shows that line congestion needs the most projections (23.0 of 72 proposals per run) and that voltage violation, battery stress and unsafe AI action need the most rejections (8.3, 8.0 and 8.0), as expected from their design. The coordinator flagged about 11.5 conflicts per run (241 in 21 runs), and the fallback rate was 0.2 %, caused by the malformed-output fault injected in unsafe AI action; a fallback decision is still validated like any other.

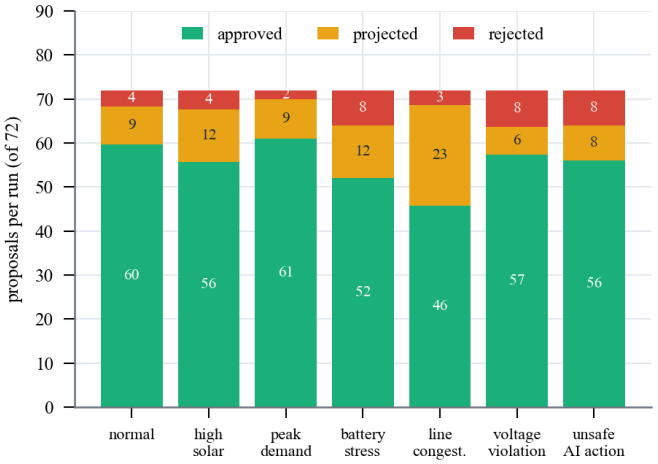


Fig. 4. Shield decisions per run (72 proposals), mean of three seeds, by scenario.

TABLE III
SAFEGRID-AI PER SCENARIO, MEAN OF 3 SEEDS (72 PROPOSALS AND 24 STEPS PER RUN)

Scenario	Approved / projected / rejected	Raw-proposal violations	Idle-grid stress steps	Violation steps after execution
normal	59.7 / 8.7 / 3.7	1.7	0.3	0.0
high solar	55.7 / 12.0 / 4.3	2.3	20.0	1.7
peak demand	61.0 / 9.0 / 2.0	14.0	12.0	8.7
battery stress	52.0 / 12.0 / 8.0	5.3	3.0	1.7
line congestion	45.7 / 23.0 / 3.3	16.0	0.0	0.0
voltage violation	57.3 / 6.3 / 8.3	12.7	6.7	4.3
unsafe ai action	56.0 / 8.0 / 8.0	12.0	2.3	1.0

Table III separates what the shield prevents from what it cannot. Raw-proposal violations are the violations the unvalidated proposals would have caused; they are largest in line congestion (16.0) and peak demand (14.0) and the shield removes all of them. Idle-grid stress is violation that exists when every agent does nothing: 20.0 of 24 steps in high solar and 12.0 in peak demand, and no proposal causes it. The shield does not repair it by design. The relief-first order and the projections nevertheless reduce it in some scenarios (high solar falls from 20.0 to 1.7 violation steps), but not in all: 8.7 of the 12.0 peak-demand steps remain in violation. Relief is a by-product, not a guarantee.

D. Single-run traces

Fig. 5 shows seed-42 runs. In line congestion the unshielded agents drive the worst line to 137.7 % loading, while the shield projects the same agents' actions to exactly 100.0 %. In voltage violation the highest bus voltage is 1.117 pu without the shield and 1.056 pu with it: slightly above 1.05 pu because stress that exists with every agent idle is not repaired. In peak demand the lowest voltage is 0.861 pu without and 0.932 pu with the shield, still below 0.95 pu for the same reason.

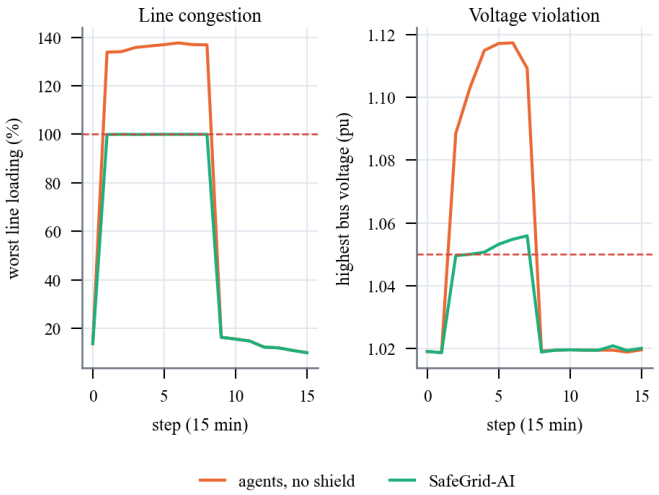


Fig. 5. Seed-42 traces, shield off and on; dashed lines are the limits.

E. Cost of safety and run-time behaviour

The shield is not free. Mean step time rises from 106 to 348 ms because each validation runs trial power flows. Renewable utilisation falls from 0.884 to 0.860 (0.607 in high solar and 0.787 in voltage violation, where PV is curtailed), while mean energy cost is 225.0 INR against 380.8 unshielded and 216.4 for the rules, about 4 % above the rule baseline. Through the live API, a dry-run validation has a median of 62.8 ms (36.6 ms for an unsafe request that is rejected, 230.0 ms when bisection is needed) and a full step 440.7 ms (95th percentile 522.2 ms). The mock provider answers in 0.17 ms on average, so these figures say nothing about the latency of a real model. Table IV lists the timings.

TABLE IV
LIVE API LATENCY ON LOCALHOST (SCENARIO UNSAFE AI ACTION, SEED 42)

Call	n	Median (ms)	95th pct. (ms)
GET /api/health	50	1.7	15.4
GET /api/grid/state	50	2.5	25.5
GET /api/scenarios	50	2.7	23.2
GET /api/agents	30	2.7	16.2
POST /api/agents/{id}/decision (dry run)	30	62.8	84.5
POST dry run, unsafe 1000 kW export (rejected)	30	36.6	80.5
POST dry run, 9 kW charge (projected by bisection)	30	230.0	262.6
POST /api/simulation/step	12	440.7	522.2

F. Explanations and audit trail in use

Every decision, approved or not, is stored with a structured explanation: the context the agent saw, the decision factors, the proposed action with its reason codes, and the shield's result. For a projected action the result lists the ten checks with their outcome (passed, failed, or not evaluated because an earlier check had already rejected the action), the violations with the quantity, the limit and the unit, and the correction from the requested power to the executed power. The dashboard presents the same record next to the live grid and the safety log lists the events with their hashes. This is a record of what was proposed, checked and changed; it is not a statement of why a language model chose an action, which is exactly what cannot be trusted to be faithful [13].

A side effect of unchecked proposals is visible in the battery. Battery throughput per run is 3.39 times the installed capacity for the unshielded agents, against 0.52 for the rules and 0.51 with the shield: over-sized charge and discharge requests cycle the batteries hard, the shield's checks cap them, and throughput returns to the level of the rule controller. This is consistent with the lower cost in Table II, although the experiment does not isolate the cause.

G. Discussion and limits

The result is narrower than the claim that motivated the project. The guarantee is that, in the simulated model, executed actions never create or worsen a violation of the configured limits; it is not a formal proof, and it does not make the grid violation-free: with every agent idle the feeder is already outside its limits in 6.3 steps per run on average, and after execution 2.48 steps per run still contain a violation, against 6.05 for the rules. A violation-resolution rate for such pre-existing stress was not measured. All agent results use the mock provider, so language-model quality and real inference latency are unmeasured; there are only three seeds, a synthetic three-node feeder, no protection, dynamics or phase unbalance, and no user study. The comparison with the base paper is qualitative: its DC microgrid, Simulink model and rule agents share no benchmark with this work. 52 of 55 unit tests pass; two grid tests fail on numeric tolerances under pandapower 3.5.6 and one WebSocket test hangs under the installed test client, while a live smoke test passes 20 of 20 checks.

Threats to validity. The mock provider is a transparent cost-aware heuristic with seeded over-sizing, so the unshielded arm's violations measure how often an over-sized or ill-timed proposal hurts, not how often a real model errs; a real model may be more or less hazardous. The scenarios were written by the authors and include faults designed to stress the shield, so the unsafe-AI-action scenario shows the shield working, not how often unsafe proposals occur in practice. With three seeds the means carry little statistical weight, and the experiment output reports the spread of each metric rather than a significance test. The 0.95 to 1.05 pu band and the 100 % loading limits are configured values, and a change of limits changes the counts.

Reproducibility. The 63-run record, the seed-42 traces and the live-API timings are stored with the code, the experiment is a single command with fixed seeds, and every number in this paper is read from those files. Wall-clock figures depend on the machine and shift between runs; the violation counts do not, because the profiles, the mock provider and the shield are deterministic for a given seed.

Operational use. Because validation takes tens to a few hundred milliseconds, the shield fits a control step of minutes. In an operator's hands the dry-run route lets a proposed set-point be checked before it is applied, the safety log shows which check failed and by how much, and the explanation attached to each decision lists the inputs and factors that led to it. None of this was evaluated with users. Before any use beyond simulation the model would have to be replaced by measured network data, protection and

dynamics would have to be modelled, and the API would need authentication.

V. CONCLUSION AND FUTURE WORK

SafeGrid-AI separates proposing from permitting: agents propose, and a deterministic shield approves, scales down or rejects each action before the simulated feeder sees it. Over 63 runs and 504 shielded steps no executed action created or worsened a limit violation, at the price of about 3.3 times the step time and some curtailed solar energy, and every decision left an explanation and a tamper-evident record. The evidence is simulated, uses a mock model and three seeds, and supports a model-relative guarantee only.

Three lessons stand out. First, a conventional rule controller was not safe either (46 violating steps), so a shield is useful whatever the proposer is. Second, the guarantee comes from checking actions, not from the model, so a stronger or weaker language model is expected to change cost and utilisation but not the safety counts; this expectation has not been tested with a live model. Third, safety has a measurable price: about 3.3 times the step time and 2.5 points of renewable utilisation (0.884 to 0.860) relative to unshielded agents.

Future work is to evaluate live language models and their latency, to measure a violation-resolution rate for pre-existing stress, to repair the two failing tolerance tests, to test larger and three-phase networks and real measurement data, to add many more seeds, and to study operator trust with a user study.

REFERENCES

[1] S. E. Eyimaya and N. Altin, "Design of multi-agent-based energy management system for DC microgrids," IEEE Access, vol. 14, pp. 39120-39133, 2026, doi: 10.1109/ACCESS.2026.3672618.

[2] H. Farhangi, "The path of the smart grid," IEEE Power Energy Mag., vol. 8, no. 1, pp. 18-28, Jan./Feb. 2010, doi: 10.1109/MPE.2009.934876.

[3] R. H. Lasseter, "MicroGrids," in Proc. IEEE Power Eng. Soc. Winter Meeting, vol. 1, 2002, pp. 305-308, doi: 10.1109/PESW.2002.985003.

[4] S. D. J. McArthur et al., "Multi-agent systems for power engineering applications - Part I: Concepts, approaches, and technical challenges," IEEE Trans. Power Syst., vol. 22, no. 4, pp. 1743-1752, Nov. 2007, doi: 10.1109/TPWRS.2007.908471.

[5] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, U.K.: Wiley, 2009.

[6] Y. Zhang, A. M. Saber, A. Youssef, and D. Kundur, "Grid-Agent: An LLM-powered multi-agent system for power grid control," arXiv:2508.05702, Aug. 2025.

[7] J. Picault and C. Goubet, "Orchestrating power grid studies with multi-agent AI and MCP servers," arXiv:2607.14158, Jul. 2026.

[8] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[9] M. Alshiekh, R. Bloem, R. Ehlers, B. Konighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conf. Artif. Intell., 2018, pp. 2669-2678.

[10] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, "Safe exploration in continuous action spaces," arXiv:1801.08757, 2018.

[11] J. Garcia and F. Fernandez, "A comprehensive survey on safe reinforcement learning," J. Mach. Learn. Res., vol. 16, pp. 1437-1480, 2015.

[12] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mane, "Concrete problems in AI safety," arXiv:1606.06565, 2016.

[13] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2023.

[14] L. Thurner et al., "pandapower - An open-source Python tool for convenient modeling, analysis, and optimization of electric power systems," IEEE Trans. Power Syst., vol. 33, no. 6, pp. 6510-6521, Nov. 2018, doi: 10.1109/TPWRS.2018.2829021.

[15] Voltage Characteristics of Electricity Supplied by Public Electricity Networks, European Standard EN 50160:2010, CENELEC, 2010.

[16] Secure Hash Standard (SHS), Federal Information Processing Standards Publication FIPS 180-4, NIST, Aug. 2015.